

P04 Recursive Rules, Fixed Points, and Provenance

Executable research capsule

PBUI research prototype

2026-08-05

Contents

1	P04 - Recursive Rules, Fixed Points, and Provenance	2
1.1	Research question	2
1.2	Why a rule layer is useful for PBUI	2
1.3	Executable language	2
1.3.1	Terms and atoms	2
1.3.2	Literals	3
1.3.3	Rules	3
1.4	Unification in rule matching	3
1.5	Fixed-point semantics	4
1.5.1	Naive evaluator	4
1.5.2	Agenda evaluator	4
1.5.3	Incremental additions	4
1.6	Stratified negation	4
1.7	Provenance	5
1.7.1	Verification	5
1.8	PBUI examples	6
1.8.1	Compatibility and selection	6
1.8.2	Action availability	6
1.8.3	Port link closure	6
1.9	Formal safety conditions	6
1.10	Use of Baader-Snyder unification theory	7
1.10.1	Rule matching produces substitutions	7
1.10.2	Generality avoids ground enumeration	7
1.10.3	Efficiency guidance	7
1.11	Tests and experiments	7
1.12	Results established by the prototype	8
1.12.1	By construction	8
1.12.2	By executable tests	8
1.12.3	Not yet proved	8
1.13	Deliberate limitations	8
1.14	Integration seam	8
1.14.1	Strategy A: P03-first	8
1.14.2	Strategy B: shared relational core	9

1 P04 - Recursive Rules, Fixed Points, and Provenance

1.1 Research question

Can PBUI applicability, compatibility, inherited scope, translator reachability, link connectivity, and action availability be expressed as a finite declarative rule program with an inspectable least-fixed-point semantics and checkable derivations?

This capsule implements a small function-free Datalog-like core with:

- ground facts and range-restricted rules;
- positive and stratified-negative literals;
- first-order syntactic unification during rule matching;
- a naive stratum-by-stratum fixed-point evaluator;
- an agenda evaluator for the positive fragment;
- incremental insertion for positive programs;
- bounded provenance alternatives;
- human-readable explanation trees;
- derivation verification;
- validation of arities, safety, finiteness restrictions, and negation strata.

1.2 Why a rule layer is useful for PBUI

Many presentation properties are relational rather than local descriptor fields:

```
selectable(context, occurrence, subject)
compatible(sourceSort, targetSort)
translatorReachable(sourceSort, targetSort)
availableAction(context, subject, action)
sameBinding(portA, portB)
```

They depend on combinations of facts:

```
selectable(C, 0, S) :-
  wants(C, T),
  presents(0, S, U),
  compatible(U, T),
  reachable(0).
```

A descriptor callback can compute one answer but does not expose the relation, its recursive dependencies, or the reason for the result. P04 makes those explicit.

1.3 Executable language

1.3.1 Terms and atoms

Rules use first-order variables and atomic constants:

```
relation("reachable", v("x"), v("y"))
fact("edge", "chart", "pipeline")
```

The fixed-point fragment is function-free. Compound terms in rule arguments are rejected. This Datalog restriction keeps the active Herbrand base finite when constants come only from the finite input program and facts.

1.3.2 Literals

```
positive(relation("edge", v("x"), v("y")))
negative(relation("archived", v("project")))
```

Negative variables must also occur in a positive body literal. Head variables must be range-restricted by positive body literals.

1.3.3 Rules

```
{
  id: "reachable-transitive",
  head: relation("reachable", v("x"), v("z")),
  body: [
    positive(relation("reachable", v("x"), v("y"))),
    positive(relation("edge", v("y"), v("z"))),
  ],
}
```

Rule identifiers are stable provenance names.

1.4 Unification in rule matching

For each positive body literal, the evaluator unifies the instantiated literal with candidate ground facts. The resulting substitution is composed with bindings accumulated from earlier literals.

Example:

```
reachable(?x, ?y)
edge(?y, ?z)
```

matched against:

```
reachable("chart", "pipeline")
edge("pipeline", "table")
```

produces:

```
?x -> "chart"
?y -> "pipeline"
?z -> "table"
```

and derives:

```
reachable("chart", "table")
```

Repeated variables are handled by unification, so:

```
pair(?x, ?x)
```

matches `pair("a", "a")` but not `pair("a", "b")`.

The term/unification code is copied into P04 rather than imported from P03. This is deliberate for the first research pass: P04 can be assessed independently and later compared with a shared unification service.

1.5 Fixed-point semantics

For a positive program, let T_P be the immediate-consequence operator on sets of ground facts. The intended result is:

$$\text{lfp}(T_P) = \bigcup_{n \geq 0} T_P^n(B),$$

where B is the extensional base database.

Because the implemented language is function-free and the active constants and predicates are finite, there are only finitely many possible ground atoms at fixed arities. Monotone iteration therefore stabilizes after finitely many fact insertions.

1.5.1 Naive evaluator

The reference evaluator repeatedly executes every rule in one stratum until no new fact appears. It then advances to the next stratum.

This evaluator is intentionally simple and serves as the semantic oracle.

1.5.2 Agenda evaluator

For positive programs, the agenda evaluator reacts to newly added facts. Rules are indexed by body predicate, and one body occurrence is anchored to the new fact while other body literals range over the complete known database.

The tests compare its final fact set with the naive evaluator.

1.5.3 Incremental additions

`IncrementalPositiveEngine` retains the derived knowledge base. Adding new base facts seeds the agenda with only those facts and newly derived consequences.

This prototype supports insertions only. Retractions require dependency counts, DRed-style maintenance, differential collections, or affected-region recomputation.

1.6 Stratified negation

Rules such as:

```
availableAction(P, "archive") :-  
    project(P),  
    not archived(P).
```

are evaluated by assigning predicates to strata. A negative dependency requires a strictly lower stratum; a positive dependency permits the same stratum.

The compiler solves constraints:

```
stratum(head) >= stratum(positiveBody)  
stratum(head) > stratum(negativeBody)
```

If repeated relaxation does not stabilize within the finite bound, the program contains recursion through negation and is rejected.

Thus the capsule has a defined semantics for stratified negation, not arbitrary nonmonotone recursion.

1.7 Provenance

Each fact stores one or more derivations:

```
type Derivation =  
  | { kind: "base"; id: string }  
  | {  
    kind: "rule";  
    ruleId: string;  
    substitution: Substitution;  
    premises: string[];  
    negativeChecks: string[];  
  };
```

A recursive explanation can show:

```
reachable(chart, table)  
  via reachable-transitive  
    reachable(chart, pipeline)  
      via edge-is-reachable  
        edge(chart, pipeline) [base]  
        edge(pipeline, table) [base]
```

The store deduplicates derivations and caps alternatives per fact to prevent an explanation experiment from consuming unbounded memory in cyclic programs.

1.7.1 Verification

verifyFactDerivation checks a stored derivation against:

- the rule named by the derivation;
- the recorded substitution;
- the instantiated head;
- the expected positive premises;

- absence of instantiated negative facts;
- recursively verifiable premise facts.

This is a small certificate checker. It does not prove that the evaluator found every derivable fact, but it prevents an optimized evaluator from inventing a fact without a valid local derivation.

1.8 PBUI examples

1.8.1 Compatibility and selection

```
compatible(S, S) :- sort(S).

selectable(C, 0, Subject) :-
    wants(C, TargetSort),
    presents(0, Subject, SourceSort),
    compatible(SourceSort, TargetSort),
    reachable(0).
```

1.8.2 Action availability

```
action(C, Subject, "inspect") :-
    selectable(C, 0, Subject),
    canInspect(C).
```

1.8.3 Port link closure

```
sameBinding(A, B) :- link(A, B).
sameBinding(B, A) :- sameBinding(A, B).
sameBinding(A, C) :- sameBinding(A, B), sameBinding(B, C).
```

The last example is appropriate for reasoning and explanation, though a production port compiler may use a specialized graph quotient algorithm for efficiency.

1.9 Formal safety conditions

The validator enforces:

1. every predicate has one arity throughout the program;
2. base facts are ground;
3. terms are function-free;
4. rule IDs are unique;
5. every head variable appears in a positive body literal;
6. every negative variable appears in a positive body literal;
7. recursion through negation is absent.

These conditions support finite evaluation and safe instantiation. They are not a complete proof of termination if future extensions add aggregates, fresh values, external functions, or infinite domains.

1.10 Use of Baader-Snyder unification theory

The chapter is relevant in three ways.

1.10.1 Rule matching produces substitutions

Rule evaluation is not merely a predicate-name join. Variables shared across body literals impose equations, and unification constructs a substitution satisfying them.

1.10.2 Generality avoids ground enumeration

A rule is written once with variables. The unifier computes the necessary instantiation from current facts. This is the same broad automation benefit that makes unification central to resolution and rewriting: the system avoids enumerating arbitrary ground substitutions before matching.

1.10.3 Efficiency guidance

The source describes term-DAG algorithms and equivalence-class approaches to unification. P04 currently uses a simple tree unifier because it is easier to audit. Future optimized matching can use shared term DAGs or union-find closure, but must be differential-tested against the current reference semantics.

The chapter's warnings about equational unification also constrain the rule language. P04 does not silently match modulo associativity, commutativity, idempotence, or application-specific equations. Those require explicitly named theory modules and may produce complete sets of unifiers rather than one principal solution.

1.11 Tests and experiments

The eleven executable tests cover:

1. least transitive closure from recursive rules;
2. agenda/reference evaluator equality;
3. rule and base-fact order independence;
4. fixed-point idempotence;
5. recursive provenance explanation and verification;
6. stratified action derivation with negation;
7. rejection of recursion through negation;
8. rejection of unsafe and function-generating rules;
9. repeated-variable unification in rule matching;
10. incremental insertion equality with full recomputation;
11. a PBUI-style selectable-occurrence/action constellation.

Run:

```
npm test
npm run demo
```

1.12 Results established by the prototype

1.12.1 By construction

- Facts are only added during fixed-point evaluation.
- Function-free range-restricted rules cannot generate fresh term structure.
- Negative dependencies are assigned to lower strata or rejected.
- Derivations name stable rules and explicit premises.

1.12.2 By executable tests

- The two positive evaluators agree on the reachability fixture.
- Rule ordering does not change the tested fixed point.
- Re-evaluating with the completed fact set is idempotent.
- Incremental insertions match complete recomputation on the fixture.
- Stored recursive derivations pass the certificate checker.

1.12.3 Not yet proved

- soundness and completeness of either evaluator;
- termination bound for all validated programs;
- correctness of the stratification solver;
- equivalence of agenda and naive evaluation for all positive programs;
- completeness of provenance alternatives;
- incremental correctness under arbitrary insertion sequences;
- correctness under deletion and negation retractions.

1.13 Deliberate limitations

1. No function symbols in rule atoms.
2. No aggregation, arithmetic, comparison predicates, or foreign functions.
3. No disjunction in one rule body; use several rules with the same head.
4. No retractions in the incremental engine.
5. Provenance alternatives are capped.
6. Negation is stratified absence from a completed lower stratum.
7. The engine is in-memory and single-threaded.
8. The derivation checker trusts the program and term implementation.
9. It does not track semantic revisions from P01-P03.

1.14 Integration seam

A later composition pass should compare two strategies:

1.14.1 Strategy A: P03-first

P03 selects candidate records. The adapter inserts those candidates as extensional P04 facts, and P04 derives recursive actions and authority.

1.14.2 Strategy B: shared relational core

A common query/rule IR compiles both nonrecursive selectors and recursive rules. P03 becomes the nonrecursive evidence-producing view of the shared engine.

The first strategy is simpler. The second may eliminate duplicated semantics but risks making the rule engine the universal abstraction. Both should be prototyped before consolidation.

1.15 Research extensions

- formalize least-fixed-point semantics and evaluator soundness in Lean;
- add semi-naive delta relations rather than fact-anchored agenda evaluation;
- implement deletions with support counts or differential dataflow;
- add semiring provenance and minimal-proof extraction;
- compile rules to indexed joins;
- investigate sorted unification and typed predicate signatures;
- introduce monotone lattice-valued attributes;
- model well-founded or stable-model semantics as explicit alternative engines;
- benchmark graph-specialized algorithms against generic rules;
- test proof certificates emitted by an optimized worker runtime.